Algorithmen und Datenstrukturen

Sommersemester 2026

Jana Grajetzki (jana.grajetzki@uni-jena.de)

Aufgabenblatt 10

Abgabe Donnerstag, den 25.6.2026, bis 12.00 Uhr. Bitte laden Sie Ihre Lösungen in Moodle als pdf-file hoch.

Begründen Sie Ihre Antworten immer, auch wenn dies nicht explizit gefordert wird.

Aufgabe 1:
Es sei $V = \{v_1, v_2, \dots v_{2k}\}$. Folgender Algorithmus ergänzt V zu einem Graphen $G = (V, E)$ durch Erzeugen einer Kantenmenge E. Er benutzt einen Disjoint-Set-Forest. MAKE-SET, FINDSET und UNION sind die aus der Vorlesung bekannten Operationen.

```
EDGES(V)
1.  for i = 1 to 2k do MAKE-SET(v_i)
2.  E := ∅
3.  for i = 1 to k do
4.     if not(FINDSET(v_2i) = FINDSET(v_2i−1)) then
5.        E := E ∪ {{v_2i, v_2i−1}}
6.        UNION(v_2i, v_2i−1)
7.  for i = 1 to 2k − 2 do
8.     if not(FINDSET(v_i) = FINDSET(v_i+2)) then
9.        E := E ∪ {{v_i, v_i+2}}
10.       UNION(v_i, v_i+2)
```

(a) Führen Sie für $k = 5$ den Algorithmus aus. Geben Sie dabei den Disjoint-Set-Forest und den Graphen nach Zeile 6 und nach jedem Durchlauf der for-Schleife in Zeilen7-10 an.

(b) Beschreiben Sie die Menge E im allgemeinen Fall.

(c) Analysieren Sie die Laufzeit dieses Algorithmus. Hinweis: Überlegen Sie sich, wieviel Zeit das FINDSET in diesem speziellen Fall tatsächlich benötigt.

(12 Punkte)

Aufgabe 2:
Ein ungerichteter Graph $G = (V, E)$ heißt bipartit, wenn es disjunkte Knotenmengen $A, B \subset V$ mit $A \cup B = V$ gibt, so dass jede Kante $e \in E$ einen Endpunkt in A und einen in B hat. Beschreiben Sie einen Algorithmus, der für einen ungerichteten Graphen testet, ob er bipartit ist. Geben Sie seine Laufzeit an. (8 Punkte)

Aufgabe 3:
Beschreiben Sie einen (möglichst effizienten) Algorithmus für folgendes Problem:

`INPUT`: Ein zusammenhängender ungerichteter Graph $G = (V, E)$.

`OUTPUT`: Ein Knoten $v \in V$, so dass $G \setminus \{v\}$ immer noch zusammenhängend ist.

Dabei bezeichnet $G \setminus \{v\}$ denjenigen Graphen, der aus G hervorgeht, wenn der Knoten v und alle zu v inzidenten Kanten gelöscht werden.
Hinweis: Überlegen Sie sich, dass es einen solchen Knoten immer geben muss und wo er im BFS-Tree zu finden ist.

(6 Punkte)

Die folgende Aufgabe ist nicht zur Korrektur abzugeben.

Aufgabe 4:
Folgendes Programm führt UNION- und FINDSET-Operationen auf einem Disjoint-Set-Forest aus. Zeichen Sie die Datenstruktur nach jeder abgearbeiteten Programmzeile. Verwenden Sie die Techniken Vereinigung nach Rang und Pfadkompression.

1. for $i = 1$ to 16 do MAKE-SET(v_i)
2. for $i = 1$ to 15 step 2 do UNION(v_i, v_{i+1})
3. for $i = 1$ to 13 step 4 do UNION(v_i, v_{i+2})
4. UNION(v_1, v_5)
5. UNION(v_{11}, v_{13})
6. UNION(v_1, v_{10})
7. FINDSET(v_2)
8. FINDSET(v_9)